

GRADEBOOK ANALYSER



***SUBJECT: PROGRAMMING FOR
PROBLEM SOLVING USING PYTHON***

COURSE CODE: ETCCPP102

MADE BY: Anshuman Sharma

CLASS: B.TECH CSE (AI ML)

SECTION: B

ROLL NO: 2501730313

INTRODUCTION

The Gradebook Analyser is a Python-based program designed to analyse student marks and generate useful statistics about their performance. It helps in calculating important values like average, median, maximum, and minimum marks from a group of students. The program can take input either manually or from a CSV file, making it flexible for different use cases. It also assigns letter grades, counts how many students fall into each grade, and clearly shows the number of passed and failed students. This makes it a simple yet effective tool to understand overall class performance.

OBJECTIVE

- To analyse student marks and compute statistical measures such as average, median, maximum, and minimum scores.
- To allow input of student data either manually or through a CSV file for convenience and reusability.
- To assign letter grades (A, B, C, D, F) based on marks and count how many students are in each grade category.
- To identify and display the list of students who passed and those who failed according to a minimum passing mark.
- To practice and strengthen understanding of Python concepts like functions, dictionaries, loops, conditionals, list comprehensions, and file handling in a real-world scenario.

WORKING DESCRIPTION

1. The program starts by importing the csv module and printing a welcome message to introduce the Gradebook Analyser. Four separate functions are defined: AVG, MEDIAN, MAX, and MIN. Each of these takes a dictionary of student names and marks and returns the respective statistic, while also safely handling the case when the dictionary is empty.
2. The main() function controls the entire flow of the program using a while True loop to repeatedly show a menu with three choices: Manual Input, CSV Input, and Exit Program. The user's choice is read inside a try-except block to handle invalid (non-numeric) inputs. If the user selects Exit, the loop breaks. If an invalid option is chosen, an error message is shown and the loop restarts, ensuring robust input validation.
3. For Manual Input (choice 1), the program asks for the number of students, then for each student it takes their name and marks and stores them in a dictionary marks_dict with the name as the key and marks as the value. A try-

except block ensures that marks are integers; if an error occurs, the loop restarts to keep the program stable.

4. For CSV Input (choice 2), the program opens a file named marks.csv using the csv.reader. It reads each row, extracts the student name and marks, converts marks to int, and stores them in marks_dict. Errors such as the file not being found or invalid data types are caught using try-except blocks, and appropriate error messages are displayed without crashing the program.
5. Once marks_dict is filled, the program prints an "Analysis_Result" section, calling AVG, MEDIAN, MAX, and MIN to show the average, median, highest, and lowest marks. Then it calculates grades for each student: A for marks ≥ 90 , B for ≥ 80 , C for ≥ 70 , D for ≥ 60 , and F for the rest. It also counts how many students fall into each grade (a, b, c, d, f) and stores the grades in another dictionary called grades.
6. The program then prints a small table showing each grade (A, B, C, D, F) and the total number of students in that category. Using list comprehensions, it creates two lists: passed_students for marks ≥ 40 and failed_students for marks < 40 . It prints how many students passed, how many failed, and lists their names in each group. Finally, it prints a detailed table of all students with their name, marks, and letter grade, followed by a separator line before returning to the main menu loop.

OUTPUT SCREENSHOTS

1. MANUAL INPUT

```
Enter choice: 1
Enter the number of students: 5
Enter name for student 1: Samarth
Enter marks for Samarth : 100
Enter name for student 2: Saurabh
Enter marks for Saurabh: 94
Enter name for student 3: Ghanishtha
Enter marks for Ghanishtha : 90
Enter name for student 4: Shiva
Enter marks for Shiva: 95
Enter name for student 5: Kaushal
Enter marks for Kaushal: 94
```

```

--- Analysis_Result ---
Average:      94.6
Median:       94
Max Score:    100
Min Score:    90

Grades        Total Students
A              5
B              0
C              0
D              0
F              0
-----
Total Passed Students: 5
Names: Samarth , Saurabh, Ghanishtha , Shiva, Kaushal
Total Failed Students: 0

```

Name	Marks	Grade
Samarth	100	A
Saurabh	94	A
Ghanishtha	90	A
Shiva	95	A
Kaushal	94	A

2. CSV INPUT

- CSV FILE TAKEN

```

marks.csv > data
1   Dev,85
2   Tanishq,92
3   Yash,96
4   Ghanishtha,66
5   Harsh,58
6   Krish,39
7   Vikram,88
8   Simran,95
9   Rahul,47
10  Meena,30

```

```
=== Gradebook Menu ===
```

```
1. Manual Input
```

```
2. CSV Input
```

```
3. Exit Program
```

```
Enter choice: 2
```

```
--- Analysis_Result ---
```

```
Average:      69.6
```

```
Median:       75.5
```

```
Max Score:    96
```

```
Min Score:    30
```

```
Grades      Total Students
```

```
A           3
```

```
B           2
```

```
C           0
```

```
D           1
```

```
F           4
```

```
-----  
Total Passed Students: 8
```

```
Names: Dev, Tanishq, Yash, Ghanishtha, Harsh, Vikram, Simran, Rahul
```

```
Total Failed Students: 2
```

```
Names: Krish, Meena
```

Name	Marks	Grade
Dev	85	B
Tanishq	92	A
Yash	96	A
Ghanishtha	66	D
Harsh	58	F
Krish	39	F
Vikram	88	B
Simran	95	A
Rahul	47	F
Meena	30	F

3. EXITING PROGRAM

```
=== Gradebook Menu ===
```

```
1. Manual Input
```

```
2. CSV Input
```

```
3. Exit Program
```

```
Enter choice: 3
```

```
Exiting Gradebook Analyzer. Goodbye!
```

```
PS C:\Users\samar\OneDrive\Desktop\Python Assignments>
```

GITHUB LINK: https://github.com/rsanshuman01-web-dev/Python_sem-1/tree/674f7b6305ee513346b338b0a37c62f256d0f2a6/gradebook_analyzer

REFLECTION

While making the Gradebook Analyser, I understood how Python can actually be used for real college work like checking class performance. It helped me get more comfortable with functions, dictionaries, loops, if-else, and error handling in one program. Adding a simple menu and giving options for manual and CSV input also taught me how to make a program easier for users to work with.

This project also cleared my doubts about average, median, highest and lowest marks, and how to calculate grades using code instead of doing it by hand. It felt nice to see the program automatically show statistics, grades, and passed/failed students. After doing this, there is more confidence and interest in building similar small tools for studies and data analysis.

CONCLUSION

The Gradebook Analyser program successfully demonstrates how Python can be used to analyse student performance in a structured and automated way. It calculates important statistics, assigns grades, and clearly shows the overall result of the class using both manual and CSV-based inputs. Through this project, it becomes easier to understand core programming concepts and how they apply to real academic scenarios like managing and analysing student marks.